Manual básico para trabajar con Markdown en Visual Studio Code

Objetivo. Aprender a crear, editar, visualizar y organizar documentación Markdown en Visual Studio Code, incluyendo títulos, tablas, imágenes, código y documentación técnica.

1. ¿Qué es Markdown?

Markdown es un lenguaje de marcado ligero que permite escribir documentos estructurados usando texto plano. Se utiliza ampliamente en proyectos de software porque es fácil de leer, versionar y mantener.

- Documentación de software
- Archivos README.md
- Documentación técnica
- Repositorios GitHub y GitLab
- Notas técnicas
- Especificaciones
- Documentación como código
- Proyectos académicos

La extensión habitual de estos archivos es:

```
.md
```

Ejemplo:

```
laboratorio_c4_logistica.md
```

2. Crear un archivo Markdown en Visual Studio Code

Paso 1. Crear una carpeta
Cree una carpeta para el proyecto, por ejemplo:
Laboratorio_C4

Paso 2. Abrir la carpeta en Visual Studio Code
Seleccione:
File → Open Folder
Luego elija la carpeta creada.

Paso 3. Crear el archivo
En el panel izquierdo seleccione New File y nombre el archivo:
laboratorio_c4_logistica.md

Manual básico para trabajar con Markdown en Visual Studio Code

3. Escribir títulos

Markdown utiliza el símbolo # para definir niveles de títulos.

```
# Título principal

## Título de segundo nivel

### Título de tercer nivel

#### Título de cuarto nivel
```

Ejemplo:

```
# Laboratorio Modelo C4

## 1. Comprensión del problema

### 1.1 Actores principales
```

Evite saltar niveles. Por ejemplo, no pase directamente de # a ### sin usar ##.

4. Escribir párrafos

Los párrafos se escriben normalmente. Para iniciar un nuevo párrafo, deje una línea vacía.

```
La plataforma LogiTrack permite gestionar la distribución de pedidos.

También permite registrar novedades.
```

5. Negrita y cursiva

Formato	Sintaxis	Resultado
Negrita	**Texto importante**	Texto importante
Cursiva	*Texto en cursiva*	Texto en cursiva
Negrita y cursiva	***Texto destacado***	Texto destacado

6. Listas

Lista con viñetas

```
- Pedido
- Cliente
- Conductor
- Vehículo
```

Manual básico para trabajar con Markdown en Visual Studio Code

Lista numerada

```
1. Analizar el problema
2. Identificar actores
3. Construir el diagrama
4. Validar el modelo
```

Subniveles

```
1. Analizar el problema
  - Identificar actores
  - Identificar sistemas externos
2. Construir el modelo
```

7. Tablas

Las tablas son útiles para documentación técnica y matrices de análisis.

```
| Elemento | Responsabilidad |
|---|---|
| Coordinador | Gestionar asignaciones |
| Conductor | Realizar entregas |
| Cliente | Consultar estado |
```

Elemento	Responsabilidad
Coordinador	Gestionar asignaciones
Conductor	Realizar entregas
Cliente	Consultar estado

8. Enlaces

```
[Texto del enlace](https://ejemplo.com)
```

Ejemplo:

```
[Documentación oficial de Markdown](https://www.markdownguide.org/)
```

9. Imágenes

Para insertar una imagen utilice:

```
![Descripción](ruta-imagen.png)
```

Ejemplo:

```
![Arquitectura del sistema](arquitectura.png)
```

Manual básico para trabajar con Markdown en Visual Studio Code

Si la imagen está dentro de una carpeta:

```
![Arquitectura](imagenes/arquitectura.png)
```

Se recomienda usar rutas relativas para que el documento siga funcionando cuando se comparta la carpeta completa del proyecto.

10. Bloques de código

Los bloques de código se delimitan con tres comillas invertidas. Puede indicarse el lenguaje para mejorar la visualización.

````
```csharp
public class Pedido
{
   public int Id { get; set; }
}
```
````

Lenguajes frecuentes csharp, java, Python, javascript, sql, json, xml

11. Código en una sola línea

```
La clase `EntregaService` gestiona las entregas.
```

12. Citas

```
> Cada nivel C4 debe ampliar el nivel anterior.
```

13. Líneas divisorias

```
---
```

Ejemplo:

```
# Nivel 1

Contenido

---

# Nivel 2
```

Manual básico para trabajar con Markdown en Visual Studio Code

14. Visualizar el archivo Markdown

En Visual Studio Code puede abrir la vista previa con:

```
Ctrl + Shift + V
```

Para ver editor y vista previa al mismo tiempo:

```
Ctrl + K, luego V
```

Esta modalidad permite trabajar con el código Markdown a la izquierda y la vista renderizada a la derecha.

15. Extensiones útiles para Markdown

Abra el panel de extensiones con:

```
Ctrl + Shift + X
```

Puede instalar extensiones orientadas a mejorar la edición y la vista previa de Markdown, por ejemplo:

- Markdown All in One
- Markdown Preview Enhanced

Estas extensiones son opcionales; el archivo .md puede editarse sin ellas.

16. Incluir diagramas en un documento Markdown

Markdown puede mostrar diagramas como imágenes. Esta es una opción estable porque no depende de que el visor interprete un lenguaje específico de diagramación.

El flujo recomendado es:

- Construir el diagrama con la herramienta de preferencia
- Exportarlo como PNG o SVG
- Guardar la imagen dentro de la carpeta del proyecto
- Referenciarla desde el archivo .md

Ejemplo:

```
![C4 Nivel 1 - Contexto](diagramas/nivel1_contexto.png)
```

Manual básico para trabajar con Markdown en Visual Studio Code

17. Organización recomendada para un laboratorio C4

Una estructura sencilla y fácil de compartir es:

```
Laboratorio_C4
│
├── laboratorio_c4_logistica.md
│
├── diagramas
│   ├── nivel1_contexto.png
│   ├── nivel2_contenedores.png
│   ├── nivel3_componentes.png
│   └── nivel4_codigo.png
│
└── imagenes
```

Así, el archivo Markdown concentra la documentación y la carpeta diagramas almacena las representaciones gráficas.

18. Uso recomendado para el laboratorio C4

```
# Laboratorio Modelo C4

## 1. Comprensión del problema

Texto...

## 2. C4 Nivel 1 - Contexto

![Nivel 1](diagramas/nivel1_contexto.png)

## 3. C4 Nivel 2 - Contenedores

![Nivel 2](diagramas/nivel2_contenedores.png)

## 4. C4 Nivel 3 - Componentes

![Nivel 3](diagramas/nivel3_componentes.png)

## 5. C4 Nivel 4 - Código

![Nivel 4](diagramas/nivel4_codigo.png)
```

Manual básico para trabajar con Markdown en Visual Studio Code

19. Documentar el uso de IA

Markdown también permite registrar prompts, resultados y decisiones del equipo.

````
## Uso de IA

### Prompt utilizado

```text
Revisa nuestro modelo C4 e identifica inconsistencias.
```

### Observaciones de la IA

- Posible inconsistencia en el Nivel 2
- Componente mal clasificado
- Dependencia innecesaria
````

20. Buenas prácticas

Mantenga una jerarquía clara de títulos

- Use tablas cuando compare elementos
- Use listas para pasos o requisitos
- Evite párrafos excesivamente largos
- Use bloques de código para código real
- Use nombres de archivo claros
- Evite espacios en nombres de archivos cuando sea posible
- Mantenga una estructura de carpetas consistente
- Use rutas relativas para imágenes y otros recursos

Ejemplo recomendado:

```
nivel1_contexto.png
```

Evite nombres ambiguos como:

```
diagrama nuevo final definitivo 2.png
```

21. Markdown y Git

Los archivos .md son archivos de texto, por lo que pueden gestionarse fácilmente con Git.

- Versionar documentación
- Comparar cambios
- Revisar quién modificó qué

Manual básico para trabajar con Markdown en Visual Studio Code

- Mantener documentación junto al código
- Automatizar documentación

Por esta razón Markdown es muy utilizado en prácticas de Documentation as Code.

22. Flujo recomendado de trabajo

1. Crear laboratorio_c4_logistica.md
2. Escribir el análisis del problema
3. Construir los diagramas C4 con la herramienta seleccionada
4. Exportar los diagramas como PNG o SVG
5. Guardar los diagramas en la carpeta diagramas
6. Referenciarlos desde el archivo .md
7. Revisar el documento completo con Ctrl + Shift + V
8. Utilizar IA para auditar el modelo
9. Documentar las decisiones finales

Markdown organiza la documentación, las imágenes permiten integrar diagramas de forma estable, Git facilita el versionamiento y la IA puede apoyar la revisión; la decisión técnica y arquitectónica sigue siendo responsabilidad del equipo.